Step 1 each DNA base rep using 2 bits. for compactness.

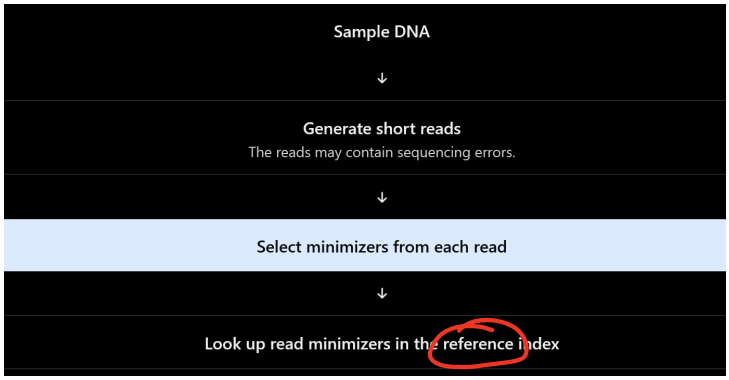
Step 2

Imagine the reference DNA contains thousands of bases. Searching and storing every possible short DNA sequence can be expensive.

Instead, the project selects a smaller set of representative sequences called **minimizers**. These help the program find matching regions efficiently when it processes sequencing reads later.

- k-mer => "k" letter split
a) split into k-mers.
b) each k-mer => HASH VALUE
c) Window of consecutive k-mers.
↓
select the one w/ MIN. # value.

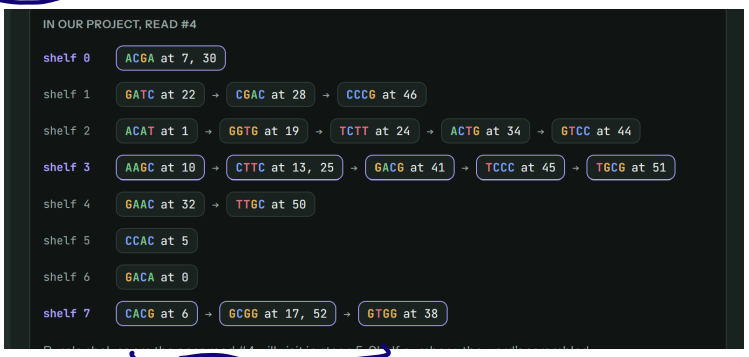
(MM) : selected set of minimizers.



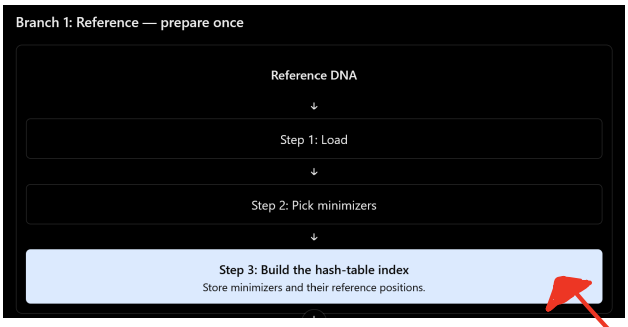
STEP 3 BUILD INDEX (Ref. branch)

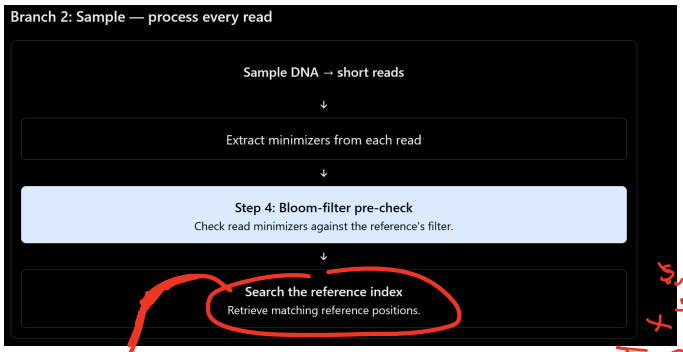
- Store selected minimizers in an "index"
↓
hash table → (old hash) % 8 => giving a number
↓
same num. stored in a BUCKET

*Only for reference not sample

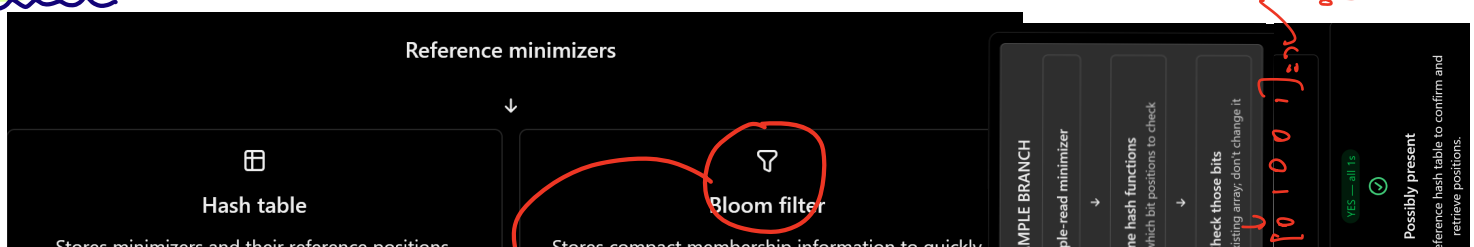


diff minimizer cnt.



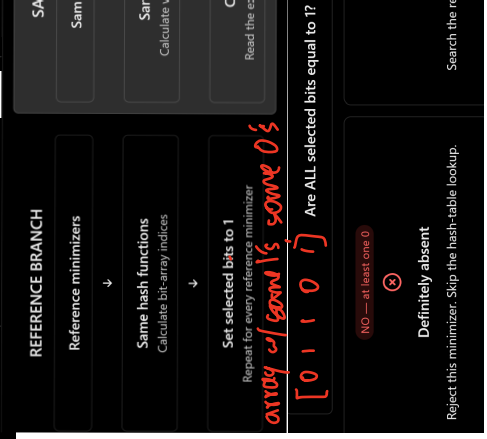


STEP 4 BLOOM FILTER *Ref. branch*



Stores minimizers and their reference positions.

Stores compact membership information to quickly reject absent minimizers.



. The Bloom filter checks whether `ACGT` might exist among the reference minimizers.

Definitely absent

The program can skip the hash-table lookup for that minimizer because it cannot occur in the inserted reference set.

Possibly present

The program proceeds to the hash-table index to check the actual word and retrieve its reference positions.

working of Bloom Filters

STEP 5 VOTE 1.

MUST-DO: count the votes in a BST
(see the next page)

- sample minimiser goes thru HT. => we get its LOC^N in REFERENCE
- Sample read has a bunch of minimizers.
  - for each, we have reference locn

READ start locn = (locn of MINIMIZER in Reference) − (posn of minimizer in read)

obtained for each minimizer in the read.

∴ multiple possible st locns obtained

VOTING (counts obtained)

Minimizer	Bloom-filter result	What happens?
A	Possibly present	Query the reference hash table
B	Definitely absent	Skip this minimizer's index lookup
C	Possibly present	Query the reference hash table
D	Possibly present	Query the reference hash table
E	Definitely absent	Skip this minimizer's index lookup

Only the individual minimizer is skipped, not the entire read.

STEP 6
- Reads st posn obtained.
- Vote counts for all st posns obtained

*TOP 3 candidates chosen in a MIN-HEAP.
- binary tree
- holds max. 3 elements.
- Lowest vote on top
- if new candidate > Lowest current,
  - ↓
  - it replaces it.

STEP 7 VERIFY MATCHING.
- Check letter for letter whether sample read actually matches the.

STEP 5 (UPDATED) VOTE WITH A BST

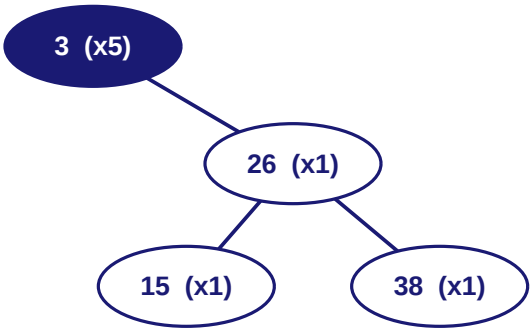
MUST-DO

Before: Vote = collect all the suggested starts in an array, sort it, and count runs of equal values.

Now: Vote = insert every suggested start into a **binary search tree (BST)**. The same start always lands on the same node, so that node's **count** is its vote count.

How it works

1. Each node stores: the start position, a count, a left pointer and a right pointer.
2. **Insert a start:** smaller than the node, go left. Larger, go right. Equal: count + 1, no new node.
3. When all hits are inserted, walk the whole tree and remember the node with the biggest count.
4. Hand every (start, count) pair to Step 6's min-heap (top 3).



Read #4: starts inserted in this order: 3, 3, 26, 3, 3, 15, 3, 38. Winner: start 3 with 5 votes.

Why this is nicer than sort + count

- Counting and ordering happen in the same structure.
- An in-order walk (left, node, right) gives 3, 15, 26, 38: already sorted.
- Insert and find cost O(h), where h is the height of the tree: about log n when it is balanced.

Watch out

If the starts arrive already sorted, a plain BST turns into a chain (like a linked list) and costs O(n). Build the plain BST first and **measure the tree height** in the benchmark. If it skews, add self-balancing (AVL) later.

What you write in C

- The node struct (start, count, left, right).
- insert(): recursive or a loop.
- An in-order walk that also tracks the biggest count.
- A function to free the whole tree.
- Bonus: Step 9's answer key can reuse the same BST code (search by position).

Benchmark link: compare Vote with a BST against sorted array + run counting on time, memory and tree height. That is a ready-made experiment for the paper.

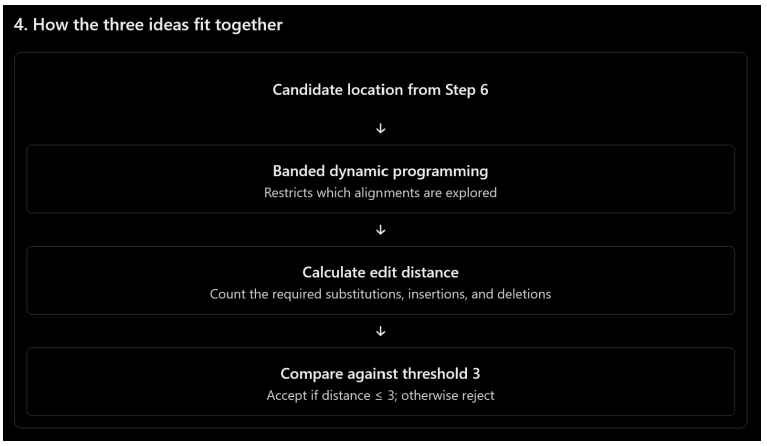
reference read.

- Might still not match even after all this filtering.

BANDED DYNAMIC PROGRAMMING

DP → A 2d matrix w/ each cell showing corresponding edit distance.
→ But checks every cell to find min. edit dist

BANDED DP | Only a BAND of cells around diagonal of matrix checked (less computation)

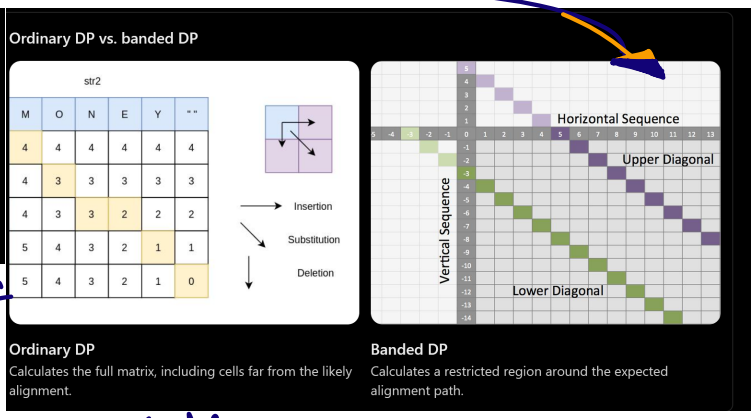


(

The complete DP matrix

Read \ Reference	–	A	G	C	T
–	0	1	2	3	4
A	1	0	1	2	3
G	2	1	0	1	2
T	3	2	1	1	1

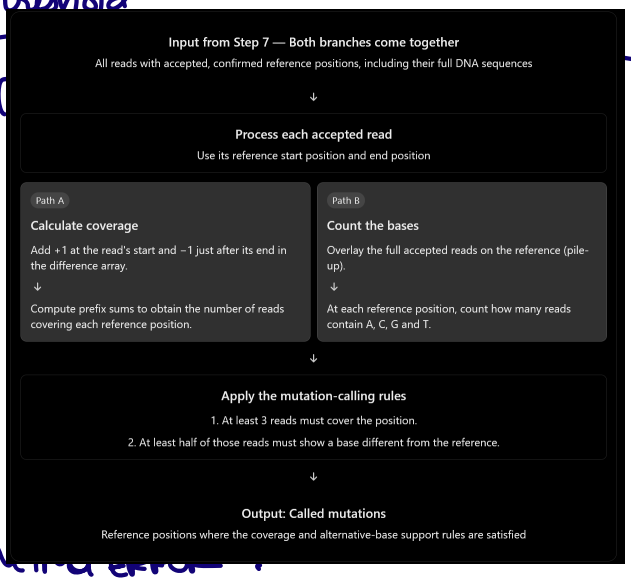
The final answer is the bottom-right cell: 1.



* decides whether a read actually present at pos" told by step 6.
* Filter out everything that has edit dist -> threshold

STEP 8 { - inp: step 7: confirmed posns.

MUTATION FINDING



* Compares all reads to reference for a ptc location.

- If huge no. of reads all report diff. letter comp to reference => MUTATION
- If only few reads show diff. value => SEQUENCING ERROR

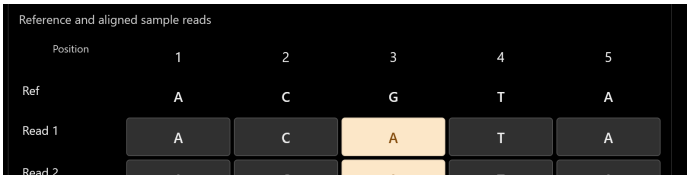
+RULE → At least (3) aligned reads cover that portion
→ >half the reads support that anomaly
} then MUTATION else ERROR

. Coverage => No. of reads that "cover" a portion of reference

For a read covering positions 2 through 4:

- Add `+1` at position 2.
- Add `-1` immediately after its end, at position 5.
- Take a prefix sum across the array.

Example: one read covers positions 2–4



Position	1	2	3	4	5	6
Difference	0	1	0	0	-1	0
Coverage	0	1	1	1	0	0

The prefix sum converts the two boundary updates into coverage at every position.

This works because the `+1` starts the coverage contribution and the `−1` cancels it just after the read ends.

Read 3	A	C	A	T	A

At reference position 3, the reference has G, while all three illustrative reads have A.

Coverage

STEP 9 EVALUATION

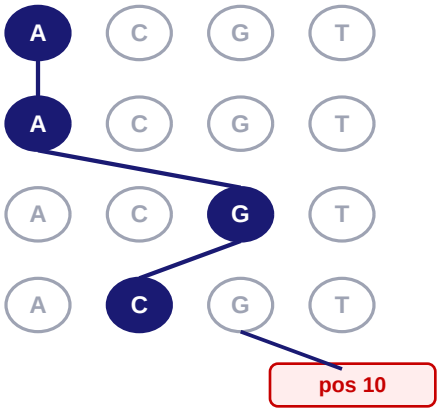
- Check step 8 prediction against actual data.
- Only possible for als. existing info abt mutation.

OPTIONAL ADD-ONS

Only after the core, including the BST, works. Order: BST (must) → trie → de Bruijn graph.

A. Trie (a 4-way tree) as a second index

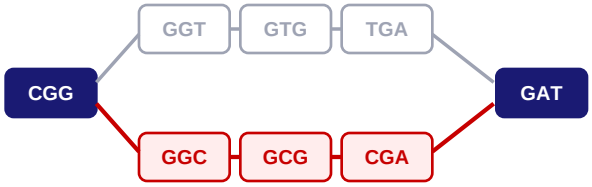
- Every node has 4 children: A, C, G, T.
- A word is a path from the root. AAGC = A → A → G → C. The last node stores the word's positions (10).
- Lookup walks down one letter at a time. It can also try one wrong letter per level, which rescues words broken by a typo.
- Benchmark it against the hash table: speed and memory.



each level picks one of 4 branches

B. De Bruijn graph cross-check (a graph)

- Nodes are 3-letter words. An edge joins two words that overlap by 2 letters (CGG → GGT).
- Build the graph from the reference and the pieces. A mutation shows up as a **bubble**: the path splits and then rejoins.
- Find the bubble with BFS or DFS. If the mutated path is backed by many pieces, it confirms the Pile-up call.
- Toy example: reference ...CGGTGATC... vs patient ...CGGCGATC... (T became C).



grey = reference path (T), red = path most pieces follow (C)